

Store Passing and Boxes

CS 350

Dr. Joseph Eremondi

March 27, 2025

Overview

Curly-Box: A Language with Mutation

Interpreting Boxes

Overview

- Objectives

Store Passing Interpreters

- Objectives
 - Understand boxes as an abstraction for mutable state

Store Passing Interpreters

- Objectives
 - Understand boxes as an abstraction for mutable state
 - Implement an interpreter for a language with mutable state

Store Passing Interpreters

- Objectives
 - Understand boxes as an abstraction for mutable state
 - Implement an interpreter for a language with mutable state
 - Where the implementation language does *not* have mutable state

Store Passing Interpreters

- Objectives
 - Understand boxes as an abstraction for mutable state
 - Implement an interpreter for a language with mutable state
 - Where the implementation language does *not* have mutable state
 - See how Racket can extend its own syntax

Store Passing Interpreters

- Objectives
 - Understand boxes as an abstraction for mutable state
 - Implement an interpreter for a language with mutable state
 - Where the implementation language does *not* have mutable state
 - See how Racket can extend its own syntax
- Key Concepts

Store Passing Interpreters

- Objectives
 - Understand boxes as an abstraction for mutable state
 - Implement an interpreter for a language with mutable state
 - Where the implementation language does *not* have mutable state
 - See how Racket can extend its own syntax
- Key Concepts
 - Mutable state

Store Passing Interpreters

- Objectives
 - Understand boxes as an abstraction for mutable state
 - Implement an interpreter for a language with mutable state
 - Where the implementation language does *not* have mutable state
 - See how Racket can extend its own syntax
- Key Concepts
 - Mutable state
 - Store data structure

Store Passing Interpreters

- Objectives
 - Understand boxes as an abstraction for mutable state
 - Implement an interpreter for a language with mutable state
 - Where the implementation language does *not* have mutable state
 - See how Racket can extend its own syntax
- Key Concepts
 - Mutable state
 - Store data structure
 - Store passing

Store Passing Interpreters

- Objectives
 - Understand boxes as an abstraction for mutable state
 - Implement an interpreter for a language with mutable state
 - Where the implementation language does *not* have mutable state
 - See how Racket can extend its own syntax
- Key Concepts
 - Mutable state
 - Store data structure
 - Store passing
 - Boxes

Store Passing Interpreters

- Objectives
 - Understand boxes as an abstraction for mutable state
 - Implement an interpreter for a language with mutable state
 - Where the implementation language does *not* have mutable state
 - See how Racket can extend its own syntax
- Key Concepts
 - Mutable state
 - Store data structure
 - Store passing
 - Boxes
 - Macros

Curly-Box: A Language with Mutation

Expressions vs. Statements

- **Expression:** a program or program fragment that is meant to be evaluated

Expressions vs. Statements

- **Expression:** a program or program fragment that is meant to be evaluated
 - Produces a value

Expressions vs. Statements

- **Expression:** a program or program fragment that is meant to be evaluated
 - Produces a value
 - e.g. Function arguments in C++/Python are expressions

Expressions vs. Statements

- **Expression:** a program or program fragment that is meant to be evaluated
 - Produces a value
 - e.g. Function arguments in C++/Python are expressions
 - Might have side-effects

Expressions vs. Statements

- **Expression:** a program or program fragment that is meant to be evaluated
 - Produces a value
 - e.g. Function arguments in C++/Python are expressions
 - Might have side-effects
- **Statement:** a program fragment that is meant to be executed

Expressions vs. Statements

- **Expression:** a program or program fragment that is meant to be evaluated
 - Produces a value
 - e.g. Function arguments in C++/Python are expressions
 - Might have side-effects
- **Statement:** a program fragment that is meant to be executed
 - `if`, loop-bodies, function bodies are all statements in Python/C++

Expressions vs. Statements

- **Expression:** a program or program fragment that is meant to be evaluated
 - Produces a value
 - e.g. Function arguments in C++/Python are expressions
 - Might have side-effects
- **Statement:** a program fragment that is meant to be executed
 - if, loop-bodies, function bodies are all statements in Python/C++
 - e.g. Had to explicitly return or use assignment to produce a value from a statement

Expressions vs. Statements

- **Expression:** a program or program fragment that is meant to be evaluated
 - Produces a value
 - e.g. Function arguments in C++/Python are expressions
 - Might have side-effects
- **Statement:** a program fragment that is meant to be executed
 - if, loop-bodies, function bodies are all statements in Python/C++
 - e.g. Had to explicitly return or use assignment to produce a value from a statement
- All of our Curly languages (and Racket) are *expression oriented languages*

Expressions vs. Statements

- **Expression:** a program or program fragment that is meant to be evaluated
 - Produces a value
 - e.g. Function arguments in C++/Python are expressions
 - Might have side-effects
- **Statement:** a program fragment that is meant to be executed
 - `if`, loop-bodies, function bodies are all statements in Python/C++
 - e.g. Had to explicitly `return` or use assignment to produce a value from a statement
- All of our Curly languages (and Racket) are *expression oriented languages*
 - No notion of statement

Expressions vs. Statements

- **Expression:** a program or program fragment that is meant to be evaluated
 - Produces a value
 - e.g. Function arguments in C++/Python are expressions
 - Might have side-effects
- **Statement:** a program fragment that is meant to be executed
 - `if`, loop-bodies, function bodies are all statements in Python/C++
 - e.g. Had to explicitly `return` or use assignment to produce a value from a statement
- All of our Curly languages (and Racket) are *expression oriented languages*
 - No notion of statement
 - Syntax consists of nested expressions

Expressions vs. Statements

- **Expression:** a program or program fragment that is meant to be evaluated
 - Produces a value
 - e.g. Function arguments in C++/Python are expressions
 - Might have side-effects
- **Statement:** a program fragment that is meant to be executed
 - `if`, loop-bodies, function bodies are all statements in Python/C++
 - e.g. Had to explicitly `return` or use assignment to produce a value from a statement
- All of our Curly languages (and Racket) are *expression oriented languages*
 - No notion of statement
 - Syntax consists of nested expressions
 - If branches: expressions

Expressions vs. Statements

- **Expression:** a program or program fragment that is meant to be evaluated
 - Produces a value
 - e.g. Function arguments in C++/Python are expressions
 - Might have side-effects
- **Statement:** a program fragment that is meant to be executed
 - `if`, loop-bodies, function bodies are all statements in Python/C++
 - e.g. Had to explicitly `return` or use assignment to produce a value from a statement
- All of our Curly languages (and Racket) are *expression oriented languages*
 - No notion of statement
 - Syntax consists of nested expressions
 - If branches: expressions
 - Function bodies: expressions

- Expression-oriented version of a statement

- Expression-oriented version of a statement
- {seq <expr> <expr>}

- Expression-oriented version of a statement
- {seq <expr> <expr>}
- Evaluates the first expression *then discards the result*

- Expression-oriented version of a statement
- {seq <expr> <expr>}
- Evaluates the first expression *then discards the result*
 - Only useful if the first expression has side effects

- Expression-oriented version of a statement
- {seq <expr> <expr>}
- Evaluates the first expression *then discards the result*
 - Only useful if the first expression has side effects
- Then evaluates the second expression

- Expression-oriented version of a statement
- {seq <expr> <expr>}
- Evaluates the first expression *then discards the result*
 - Only useful if the first expression has side effects
- Then evaluates the second expression
- The value of the second expression is the value of the whole seq expression

Boxes: Mutation in Functional Languages

- Kind of like C++ References

Boxes: Mutation in Functional Languages

- Kind of like C++ References
- Pointers, but with no notion of pointer arithmetic

Boxes: Mutation in Functional Languages

- Kind of like C++ References
- Pointers, but with no notion of pointer arithmetic
- A box denotes **a location in the store**

Boxes: Mutation in Functional Languages

- Kind of like C++ References
- Pointers, but with no notion of pointer arithmetic
- A box denotes **a location in the store**
 - Store: the current state of mutable values

Boxes: Mutation in Functional Languages

- Kind of like C++ References
- Pointers, but with no notion of pointer arithmetic
- A box denotes **a location in the store**
 - Store: the current state of mutable values
 - e.g. a model of memory/RAM

Boxes: Mutation in Functional Languages

- Kind of like C++ References
- Pointers, but with no notion of pointer arithmetic
- A box denotes **a location in the store**
 - Store: the current state of mutable values
 - e.g. a model of memory/RAM
- Copies of a box refer to the same part of memory

Boxes: Mutation in Functional Languages

- Kind of like C++ References
- Pointers, but with no notion of pointer arithmetic
- A box denotes **a location in the store**
 - Store: the current state of mutable values
 - e.g. a model of memory/RAM
- Copies of a box refer to the same part of memory
 - So changes to that part are seen by *all* copies of that box

Boxes in Curly-Box

- `{box <expr>}`

Boxes in Curly-Box

- {box <expr>}
 - Allocates a new box in the store with the

Boxes in Curly-Box

- {box <expr>}
 - Allocates a new box in the store with the
 - Like C++ `void* x = new int; *x = expr;`

Boxes in Curly-Box

- {box <expr>}
 - Allocates a new box in the store with the
 - Like C++ `void* x = new int; *x = expr;`
- {unbox <expr>}

Boxes in Curly-Box

- {box <expr>}
 - Allocates a new box in the store with the
 - Like C++ `void* x = new int; *x = expr;`
- {unbox <expr>}
 - Evaluates the expression to a box

Boxes in Curly-Box

- {box <expr>}
 - Allocates a new box in the store with the
 - Like C++ `void* x = new int; *x = expr;`
- {unbox <expr>}
 - Evaluates the expression to a box
 - dynamic type error if not a box

Boxes in Curly-Box

- `{box <expr>}`
 - Allocates a new box in the store with the
 - Like C++ `void* x = new int; *x = expr;`
- `{unbox <expr>}`
 - Evaluates the expression to a box
 - dynamic type error if not a box
 - Produces the value for that box's location in memory

Boxes in Curly-Box

- {box <expr>}
 - Allocates a new box in the store with the
 - Like C++ `void* x = new int; *x = expr;`
- {unbox <expr>}
 - Evaluates the expression to a box
 - dynamic type error if not a box
 - Produces the value for that box's location in memory
 - Like C++ `*x;`

Boxes in Curly-Box

- {box <expr>}
 - Allocates a new box in the store with the
 - Like C++ `void* x = new int; *x = expr;`
- {unbox <expr>}
 - Evaluates the expression to a box
 - dynamic type error if not a box
 - Produces the value for that box's location in memory
 - Like C++ `*x;`
- {set-box! <expr> <expr>}

Boxes in Curly-Box

- {box <expr>}
 - Allocates a new box in the store with the
 - Like C++ `void* x = new int; *x = expr;`
- {unbox <expr>}
 - Evaluates the expression to a box
 - dynamic type error if not a box
 - Produces the value for that box's location in memory
 - Like C++ `*x;`
- {set-box! <expr> <expr>}
 - The ! indicates that the form has a side effect

Boxes in Curly-Box

- `{box <expr>}`
 - Allocates a new box in the store with the
 - Like C++ `void* x = new int; *x = expr;`
- `{unbox <expr>}`
 - Evaluates the expression to a box
 - dynamic type error if not a box
 - Produces the value for that box's location in memory
 - Like C++ `*x;`
- `{set-box! <expr> <expr>}`
 - The `!` indicates that the form has a side effect
 - Just a convention, `!` is a normal character in Racket/Curly

Boxes in Curly-Box

- `{box <expr>}`
 - Allocates a new box in the store with the
 - Like C++ `void* x = new int; *x = expr;`
- `{unbox <expr>}`
 - Evaluates the expression to a box
 - dynamic type error if not a box
 - Produces the value for that box's location in memory
 - Like C++ `*x;`
- `{set-box! <expr> <expr>}`
 - The `!` indicates that the form has a side effect
 - Just a convention, `!` is a normal character in Racket/Curly
 - Evaluates the first expression to a box

Boxes in Curly-Box

- `{box <expr>}`
 - Allocates a new box in the store with the
 - Like C++ `void* x = new int; *x = expr;`
- `{unbox <expr>}`
 - Evaluates the expression to a box
 - dynamic type error if not a box
 - Produces the value for that box's location in memory
 - Like C++ `*x;`
- `{set-box! <expr> <expr>}`
 - The `!` indicates that the form has a side effect
 - Just a convention, `!` is a normal character in Racket/Curly
 - Evaluates the first expression to a box
 - Evaluates the second expression

Boxes in Curly-Box

- {box <expr>}
 - Allocates a new box in the store with the
 - Like C++ `void* x = new int; *x = expr;`
- {unbox <expr>}
 - Evaluates the expression to a box
 - dynamic type error if not a box
 - Produces the value for that box's location in memory
 - Like C++ `*x;`
- {set-box! <expr> <expr>}
 - The ! indicates that the form has a side effect
 - Just a convention, ! is a normal character in Racket/Curly
 - Evaluates the first expression to a box
 - Evaluates the second expression
 - Stores that value in the box's location in memory

Boxes in Curly-Box

- `{box <expr>}`
 - Allocates a new box in the store with the
 - Like C++ `void* x = new int; *x = expr;`
- `{unbox <expr>}`
 - Evaluates the expression to a box
 - dynamic type error if not a box
 - Produces the value for that box's location in memory
 - Like C++ `*x;`
- `{set-box! <expr> <expr>}`
 - The `!` indicates that the form has a side effect
 - Just a convention, `!` is a normal character in Racket/Curly
 - Evaluates the first expression to a box
 - Evaluates the second expression
 - Stores that value in the box's location in memory
 - Produces that value as the result

Boxes in Curly-Box

- `{box <expr>}`
 - Allocates a new box in the store with the
 - Like C++ `void* x = new int; *x = expr;`
- `{unbox <expr>}`
 - Evaluates the expression to a box
 - dynamic type error if not a box
 - Produces the value for that box's location in memory
 - Like C++ `*x;`
- `{set-box! <expr> <expr>}`
 - The `!` indicates that the form has a side effect
 - Just a convention, `!` is a normal character in Racket/Curly
 - Evaluates the first expression to a box
 - Evaluates the second expression
 - Stores that value in the box's location in memory
 - Produces that value as the result
 - Other design choices are possible for the result

Boxes in Curly-Box

- `{box <expr>}`
 - Allocates a new box in the store with the
 - Like C++ `void* x = new int; *x = expr;`
- `{unbox <expr>}`
 - Evaluates the expression to a box
 - dynamic type error if not a box
 - Produces the value for that box's location in memory
 - Like C++ `*x;`
- `{set-box! <expr> <expr>}`
 - The `!` indicates that the form has a side effect
 - Just a convention, `!` is a normal character in Racket/Curly
 - Evaluates the first expression to a box
 - Evaluates the second expression
 - Stores that value in the box's location in memory
 - Produces that value as the result
 - Other design choices are possible for the result
 - Like C++ `*x = expr;`

Example

Example

```
{let1 x {box 3}
  {let1 y x
    {seq {set-box! y 10}
          {unbox x}}}}
```

- Allocates a new box, with the value 3

Example

```
{let1 x {box 3}
  {let1 y x
    {seq {set-box! y 10}
          {unbox x}}}}
```

- Allocates a new box, with the value 3
- The variable x is given the value of that box

Example

```
{let1 x {box 3}
  {let1 y x
    {seq {set-box! y 10}
          {unbox x}}}}
```

- Allocates a new box, with the value 3
- The variable x is given the value of that box
- The variable y is given the value of x

Example

```
{let1 x {box 3}
  {let1 y x
    {seq {set-box! y 10}
          {unbox x}}}}}
```

- Allocates a new box, with the value 3
- The variable x is given the value of that box
- The variable y is given the value of x
 - e.g. the value of the box

Example

```
{let1 x {box 3}
  {let1 y x
    {seq {set-box! y 10}
          {unbox x}}}}
```

- Allocates a new box, with the value 3
- The variable x is given the value of that box
- The variable y is given the value of x
 - e.g. the value of the box
- We write 10 to *the location that y points to*

Example

```
{let1 x {box 3}
  {let1 y x
    {seq {set-box! y 10}
          {unbox x}}}}
```

- Allocates a new box, with the value 3
- The variable x is given the value of that box
- The variable y is given the value of x
 - e.g. the value of the box
- We write 10 to *the location that y points to*
 - This is the same location that x points to

Example

```
{let1 x {box 3}
  {let1 y x
    {seq {set-box! y 10}
          {unbox x}}}}
```

- Allocates a new box, with the value 3
- The variable x is given the value of that box
- The variable y is given the value of x
 - e.g. the value of the box
- We write 10 to *the location that y points to*
 - This is the same location that x points to
- We produce the value from wherever x points

Example

```
{let1 x {box 3}
  {let1 y x
    {seq {set-box! y 10}
          {unbox x}}}}
```

- Allocates a new box, with the value 3
- The variable x is given the value of that box
- The variable y is given the value of x
 - e.g. the value of the box
- We write 10 to *the location that y points to*
 - This is the same location that x points to
- We produce the value from wherever x points
 - 10, since we updated its value

Boxes are First-Class Values

- Boxes are a new type of value

Boxes are First-Class Values

- Boxes are a new type of value
 - Just like pointers are values in C++

Boxes are First-Class Values

- Boxes are a new type of value
 - Just like pointers are values in C++
- Functions can create, take, return boxes

Boxes are First-Class Values

- Boxes are a new type of value
 - Just like pointers are values in C++
- Functions can create, take, return boxes
 - e.g. pass a box to a function

Boxes are First-Class Values

- Boxes are a new type of value
 - Just like pointers are values in C++
- Functions can create, take, return boxes
 - e.g. pass a box to a function
 - do `set-box!` on it

Boxes are First-Class Values

- Boxes are a new type of value
 - Just like pointers are values in C++
- Functions can create, take, return boxes
 - e.g. pass a box to a function
 - do `set-box!` on it
 - After function, new value will be seen

Example

Example

```
{let1 {double! {lam x
                {set-box x
                  {* 2 {unbox x}}}}}
  {let1 {b {box 3}}}
    {seq {double! b}
         {unbox b}}}
```

- Result is 6

Example

```
{let1 {double! {lam x
                {set-box x
                  {* 2 {unbox x}}}}}
  {let1 {b {box 3}}
    {seq {double! b}
         {unbox b}}}}
```

- Result is 6
 - Calling `double!` gave the *location* containing 3, not a copy of the value 3

Example

```
{let1 {double! {lam x
                {set-box x
                  {* 2 {unbox x}}}}}
  {let1 {b {box 3}}
    {seq {double! b}
         {unbox b}}}}
```

- Result is 6
 - Calling `double!` gave the *location* containing 3, not a copy of the value 3
 - So the call changed what was at `b`'s location

Example

```
{let1 {double! {lam x
                {set-box x
                  {* 2 {unbox x}}}}}
  {let1 {b {box 3}}
    {seq {double! b}
         {unbox b}}}}
```

- Result is 6
 - Calling `double!` gave the *location* containing 3, not a copy of the value 3
 - So the call changed what was at `b`'s location
 - So unboxing `b` after had the new value

Interpreting Boxes

- We *could* just use Racket mutation to implement boxes

- We *could* just use Racket mutation to implement boxes
 - But we're not going to

- We *could* just use Racket mutation to implement boxes
 - But we're not going to
- Recall: when we define a functional language, we get equations about the functions we define

- We *could* just use Racket mutation to implement boxes
 - But we're not going to
- Recall: when we define a functional language, we get equations about the functions we define
 - e.g. `(interp (plusE x y))` is *mathematically equal* to `(+ (interp x) (interp y))`

- We *could* just use Racket mutation to implement boxes
 - But we're not going to
- Recall: when we define a functional language, we get equations about the functions we define
 - e.g. `(interp (plusE x y))` is *mathematically equal* to `(+ (interp x) (interp y))`
- Our interpreters aren't just implementations

- We *could* just use Racket mutation to implement boxes
 - But we're not going to
- Recall: when we define a functional language, we get equations about the functions we define
 - e.g. `(interp (plusE x y))` is *mathematically equal* to `(+ (interp x) (interp y))`
- Our interpreters aren't just implementations
 - They're *specifications* for the language

- We *could* just use Racket mutation to implement boxes
 - But we're not going to
- Recall: when we define a functional language, we get equations about the functions we define
 - e.g. `(interp (plusE x y))` is *mathematically equal* to `(+ (interp x) (interp y))`
- Our interpreters aren't just implementations
 - They're *specifications* for the language
 - Semantics

- We *could* just use Racket mutation to implement boxes
 - But we're not going to
- Recall: when we define a functional language, we get equations about the functions we define
 - e.g. `(interp (plusE x y))` is *mathematically equal* to `(+ (interp x) (interp y))`
- Our interpreters aren't just implementations
 - They're *specifications* for the language
 - Semantics
 - Equations we can use to reason about program behaviour

- We *could* just use Racket mutation to implement boxes
 - But we're not going to
- Recall: when we define a functional language, we get equations about the functions we define
 - e.g. `(interp (plusE x y))` is *mathematically equal* to `(+ (interp x) (interp y))`
- Our interpreters aren't just implementations
 - They're *specifications* for the language
 - Semantics
 - Equations we can use to reason about program behaviour
- By using a purely functional approach, we define a semantics for Curly, even if it has mutation

- We *could* just use Racket mutation to implement boxes
 - But we're not going to
- Recall: when we define a functional language, we get equations about the functions we define
 - e.g. $(\text{interp } (\text{plusE } x \ y))$ is *mathematically equal* to $(+ (\text{interp } x) (\text{interp } y))$
- Our interpreters aren't just implementations
 - They're *specifications* for the language
 - Semantics
 - Equations we can use to reason about program behaviour
- By using a purely functional approach, we define a semantics for Curly, even if it has mutation
- The semantics is informal, but still a useful tool for *understanding* stateful programs and languages

New Abstract Syntax

New Abstract Syntax

```
(define-type Exp
  ...
  ;; Create a new box with the given value
  (boxE [e : Exp])
  ;; Write the given value to the given box
  (set-box!E [theBox : Exp]
             [valToWrite : Exp])
  (unboxE [theBox : Exp])
  (seqE [eFirst : Exp]
        [eSecond : Exp])
)
```

- Data structure modelling memory

- Data structure modelling memory

- Data structure modelling memory

```
(define-type-alias Location Number)
```

```
(define-type-alias Store (AssocList Location Value))
```

```
(define mt-store empty)
```

- Store has the same interface as Environment

- Data structure modelling memory

```
(define-type-alias Location Number)
```

```
(define-type-alias Store (AssocList Location Value))
```

```
(define mt-store empty)
```

- Store has the same interface as Environment
 - But keys are locations, not symbols

- Data structure modelling memory

```
(define-type-alias Location Number)
```

```
(define-type-alias Store (AssocList Location Value))
```

```
(define mt-store empty)
```

- Store has the same interface as Environment
 - But keys are locations, not symbols
 - We store locations as numbers, but that's an implementation detail

Store Operations: override-store

Store Operations: override-store

```
(define override-store  
  : (Location Value Store -> Store)  
  extend)
```

- Creates a *new store* that has the given value at the given location

Store Operations: override-store

```
(define override-store  
  : (Location Value Store -> Store)  
  extend)
```

- Creates a *new store* that has the given value at the given location
 - If something was in that location, the new value overwrites it

Store Operations: override-store

```
(define override-store  
  : (Location Value Store -> Store)  
  extend)
```

- Creates a *new store* that has the given value at the given location
 - If something was in that location, the new value overwrites it
 - In our implementation, this happens by adding the new value to the start of the list, so a search will always find it first

Store Operations: override-store

```
(define override-store  
  : (Location Value Store -> Store)  
  extend)
```

- Creates a *new store* that has the given value at the given location
 - If something was in that location, the new value overwrites it
 - In our implementation, this happens by adding the new value to the start of the list, so a search will always find it first
- Original store is unchanged

Store Operations: override-store

```
(define override-store  
  : (Location Value Store -> Store)  
  extend)
```

- Creates a *new store* that has the given value at the given location
 - If something was in that location, the new value overwrites it
 - In our implementation, this happens by adding the new value to the start of the list, so a search will always find it first
- Original store is unchanged
 - Purely functional *implementation language*

Store Operations: lookup

- `lookup : (Location Store -> Value)`

Store Operations: lookup

- `lookup : (Location Store -> Value)`
- Get the value at the given location

Store Operations: lookup

- `lookup : (Location Store -> Value)`
- Get the value at the given location
 - Finds the first one in the list e.g. from the most recent `override-store`

Store Operations: new-loc

Store Operations: new-loc

```
(define (new-loc [sto : Store])  
  : Location  
  (+ 1 (max-loc sto)))  
  
(define (max-loc [sto : Store])  
  : Location  
  (foldr max 0 (map pairFst sto)))
```

- Get a new Location that does not yet have a value in the store

Store Operations: new-loc

```
(define (new-loc [sto : Store])  
  : Location  
  (+ 1 (max-loc sto)))  
  
(define (max-loc [sto : Store])  
  : Location  
  (foldr max 0 (map pairFst sto)))
```

- Get a new Location that does not yet have a value in the store
 - max-loc is a helper that enables this

Store Operations: new-loc

```
(define (new-loc [sto : Store])  
  : Location  
  (+ 1 (max-loc sto)))  
  
(define (max-loc [sto : Store])  
  : Location  
  (foldr max 0 (map pairFst sto)))
```

- Get a new Location that does not yet have a value in the store
 - max-loc is a helper that enables this
 - Gets the list of all locations in the store

Store Operations: new-loc

```
(define (new-loc [sto : Store])  
  : Location  
  (+ 1 (max-loc sto)))  
  
(define (max-loc [sto : Store])  
  : Location  
  (foldr max 0 (map pairFst sto)))
```

- Get a new Location that does not yet have a value in the store
 - max-loc is a helper that enables this
 - Gets the list of all locations in the store
 - Iterates (folds) to find the largest

Store Operations: new-loc

```
(define (new-loc [sto : Store])  
  : Location  
  (+ 1 (max-loc sto)))  
  
(define (max-loc [sto : Store])  
  : Location  
  (foldr max 0 (map pairFst sto)))
```

- Get a new Location that does not yet have a value in the store
 - max-loc is a helper that enables this
 - Gets the list of all locations in the store
 - Iterates (folds) to find the largest
- Used with override-store to extend a store to a new one with an additional location

A Store Passing Interpreter

- Generative recursion: added an extra parameter to our function to simulate something being updated

A Store Passing Interpreter

- Generative recursion: added an extra parameter to our function to simulate something being updated
- To interpret a language with mutable state, we'll do something similar

A Store Passing Interpreter

- Generative recursion: added an extra parameter to our function to simulate something being updated
- To interpret a language with mutable state, we'll do something similar
 - Add a Store parameter to `interp`

A Store Passing Interpreter

- Generative recursion: added an extra parameter to our function to simulate something being updated
- To interpret a language with mutable state, we'll do something similar
 - Add a `Store` parameter to `interp`
 - Make `interp` produce both a `Value` and an updated `Store`

A Store Passing Interpreter

- Generative recursion: added an extra parameter to our function to simulate something being updated
- To interpret a language with mutable state, we'll do something similar
 - Add a `Store` parameter to `interp`
 - Make `interp` produce both a `Value` and an updated `Store`
 - And take `Store` as argument

A Store Passing Interpreter

- Generative recursion: added an extra parameter to our function to simulate something being updated
- To interpret a language with mutable state, we'll do something similar
 - Add a `Store` parameter to `interp`
 - Make `interp` produce both a `Value` and an updated `Store`
 - And take `Store` as argument

A Store Passing Interpreter

- Generative recursion: added an extra parameter to our function to simulate something being updated
- To interpret a language with mutable state, we'll do something similar
 - Add a Store parameter to `interp`
 - Make `interp` produce both a Value and an updated Store
 - And take Store as argument

```
(define-type-alias Result
  (Value * Store))

(define (v*s [v : Value]
            [s : Store]) : Result
  (pair v s))

(define (interp [env : Env]
               [e : Exp]
               [sto : Store]) : Result ....)
```

Updating The Interpreter: Plus

Updating The Interpreter: Plus

```
(define (interp [env : Env]
                [e : Exp]
                [sto : Store]) : Result
  (type-case Exp e
    [(Plus l r)
     (let* ([lresult (interp env l sto)]
            [v-l (pairFst lresult)] ;; Value
            [sto-l (pairSnd lresult)]
            [rresult (interp env l sto-l)] ;; sto-l, NOT sto
            [v-r (pairRst rresult)]
            [sto-r (pairSnd rresult)]) ;; sto-l, NOT sto
       (v*s (lift-binop + v-l v-r)
            sto-r))] ;; Ending store is the state after evaluating r
```

- Interp the left, getting its value and store

Updating The Interpreter: Plus

```
(define (interp [env : Env]
                [e : Exp]
                [sto : Store]) : Result
  (type-case Exp e
    [(Plus l r)
     (let* ([lresult (interp env l sto)]
            [v-l (pairFst lresult)] ;; Value
            [sto-l (pairSnd lresult)]
            [rresult (interp env l sto-l)] ;; sto-l, NOT sto
            [v-r (pairRst rresult)]
            [sto-r (pairSnd rresult)]) ;; sto-l, NOT sto
       (v*s (lift-binop + v-l v-r)
            sto-r))] ;; Ending store is the state after evaluating r
```

- Interp the left, getting its value and store
- Interp the right *using the new store from the left*

Updating The Interpreter: Plus

```
(define (interp [env : Env]
                [e : Exp]
                [sto : Store]) : Result
  (type-case Exp e
    [(Plus l r)
     (let* ([lresult (interp env l sto)]
            [v-l (pairFst lresult)];; Value
            [sto-l (pairSnd lresult)]
            [rresult (interp env l sto-l)] ;; sto-l, NOT sto
            [v-r (pairRst rresult)]
            [sto-r (pairSnd rresult)]);; sto-l, NOT sto
          (v*s (lift-binop + v-l v-r)
               sto-r))] ;; Ending store is the state after evaluating r
```

- Interp the left, getting its value and store
- Interp the right *using the new store from the left*
 - Note the sequencing: the store gets threaded through

Updating The Interpreter: Plus

```
(define (interp [env : Env]
                [e : Exp]
                [sto : Store]) : Result
  (type-case Exp e
    [(Plus l r)
     (let* ([lresult (interp env l sto)]
            [v-l (pairFst lresult)] ;; Value
            [sto-l (pairSnd lresult)]
            [rresult (interp env l sto-l)] ;; sto-l, NOT sto
            [v-r (pairRst rresult)]
            [sto-r (pairSnd rresult)]);; sto-l, NOT sto
      (v*s (lift-binop + v-l v-r)
            sto-r))] ;; Ending store is the state after evaluating r
```

- Interp the left, getting its value and store
- Interp the right *using the new store from the left*
 - Note the sequencing: the store gets threaded through
- Do the addition, *then return the resulting store from the right*

Updating The Interpreter: Plus

```
(define (interp [env : Env]
                [e : Exp]
                [sto : Store]) : Result
  (type-case Exp e
    [(Plus l r)
     (let* ([lresult (interp env l sto)]
            [v-l (pairFst lresult)] ;; Value
            [sto-l (pairSnd lresult)]
            [rresult (interp env l sto-l)] ;; sto-l, NOT sto
            [v-r (pairRst rresult)]
            [sto-r (pairSnd rresult)]) ;; sto-l, NOT sto
       (v*s (lift-binop + v-l v-r)
            sto-r))] ;; Ending store is the state after evaluating r
```

- Interp the left, getting its value and store
- Interp the right *using the new store from the left*
 - Note the sequencing: the store gets threaded through
- Do the addition, *then return the resulting store from the right*
 - Design choice: which operand we eval first makes a difference

Aside: Racket Macros

- Racket lets you define your own syntactic sugar

Aside: Racket Macros

- Racket lets you define your own syntactic sugar
 - Add or rewrite Racket syntax

Aside: Racket Macros

- Racket lets you define your own syntactic sugar
 - Add or rewrite Racket syntax
- These are called macros

Aside: Racket Macros

- Racket lets you define your own syntactic sugar
 - Add or rewrite Racket syntax
- These are called macros
- C++ has *textual macros*

Aside: Racket Macros

- Racket lets you define your own syntactic sugar
 - Add or rewrite Racket syntax
- These are called macros
- C++ has *textual macros*
 - Just inserts a string into the text of your source file

Aside: Racket Macros

- Racket lets you define your own syntactic sugar
 - Add or rewrite Racket syntax
- These are called macros
- C++ has *textual macros*
 - Just inserts a string into the text of your source file
- Racket has *syntactic macros*

Aside: Racket Macros

- Racket lets you define your own syntactic sugar
 - Add or rewrite Racket syntax
- These are called macros
- C++ has *textual macros*
 - Just inserts a string into the text of your source file
- Racket has *syntactic macros*
 - Actually lets you write functions that run at compile-time and inspect the syntax given

Aside: Racket Macros

- Racket lets you define your own syntactic sugar
 - Add or rewrite Racket syntax
- These are called macros
- C++ has *textual macros*
 - Just inserts a string into the text of your source file
- Racket has *syntactic macros*
 - Actually lets you write functions that run at compile-time and inspect the syntax given
- `define-type` and `type-case` are both Racket macros

Aside: Racket Macros

- Racket lets you define your own syntactic sugar
 - Add or rewrite Racket syntax
- These are called macros
- C++ has *textual macros*
 - Just inserts a string into the text of your source file
- Racket has *syntactic macros*
 - Actually lets you write functions that run at compile-time and inspect the syntax given
- `define-type` and `type-case` are both Racket macros
- The entire Flit type system is implemented with Racket macros

A Macro for Interpreting with Stores

A Macro for Interpreting with Stores

```
(define-syntax with
  (syntax-rules ()
    [(with body) body]
    [(with [(v-id sto-id) call] [(v-ids sto-ids) calls]... body)
     (let* ([vs call]
              [v-id (pairFst vs)]
              [sto-id (pairSnd vs)])
       (with [(v-ids sto-ids) calls]... body)))]))
```

- I won't ask you to write a macro on an exam

A Macro for Interpreting with Stores

```
(define-syntax with
  (syntax-rules ()
    [(with body) body]
    [(with [(v-id sto-id) call] [(v-ids sto-ids) calls]... body)
     (let* ([vs call]
             [v-id (pairFst vs)]
             [sto-id (pairSnd vs)])
       (with [(v-ids sto-ids) calls]... body))]))
```

- I won't ask you to write a macro on an exam
- I might check whether you know that

A Macro for Interpreting with Stores

```
(define-syntax with
  (syntax-rules ()
    [(with body) body]
    [(with [(v-id sto-id) call] [(v-ids sto-ids) calls]... body)
      (let* ([vs call]
               [v-id (pairFst vs)]
               [sto-id (pairSnd vs)])
        (with [(v-ids sto-ids) calls]... body)))]))
```

- I won't ask you to write a macro on an exam
- I might check whether you know that
 - Macros extend the syntax of a language using the language itself

A Macro for Interpreting with Stores

```
(define-syntax with
  (syntax-rules ()
    [(with body) body]
    [(with [(v-id sto-id) call] [(v-ids sto-ids) calls]... body)
     (let* ([vs call]
             [v-id (pairFst vs)]
             [sto-id (pairSnd vs)])
       (with [(v-ids sto-ids) calls]... body)))]))
```

- I won't ask you to write a macro on an exam
- I might check whether you know that
 - Macros extend the syntax of a language using the language itself
 - Macros are code that is run at compile-time

Why Bother With a Macro

- Makes code shorter

Why Bother With a Macro

- Makes code shorter
- Makes intent clearer

Why Bother With a Macro

- Makes code shorter
- Makes intent clearer
 - “Interpret this code, then give a name to the value and store that result”

Using Our Macro

Using Our Macro

```
(define (interp [env : Env]
               [e : Exp]
               [sto : Store]) : Result
  (type-case Exp e
    [(plusE l r)
     (with [(v-l sto-l) (interp env l sto)]
           [(v-r sto-r) (interp env r sto-l)]
           (v*s (lift-binop + v-l v-r)
                 sto-r))])
```

- Much more succinct: call `interp`, and immediately give a name to the store and value we get as a result

Using Our Macro

```
(define (interp [env : Env]
               [e : Exp]
               [sto : Store]) : Result
  (type-case Exp e
    [(plusE l r)
     (with [(v-l sto-l) (interp env l sto)]
           [(v-r sto-r) (interp env r sto-l)]
           (v*s (lift-binop + v-l v-r)
                 sto-r))])
```

- Much more succinct: call `interp`, and immediately give a name to the store and value we get as a result
- Updating all the other cases is similar

Using Our Macro

```
(define (interp [env : Env]
                [e : Exp]
                [sto : Store]) : Result
  (type-case Exp e
    [(plusE l r)
     (with [(v-l sto-l) (interp env l sto)]
           [(v-r sto-r) (interp env r sto-l)]
           (v*s (lift-binop + v-l v-r)
                 sto-r)))])
```

- Much more succinct: call `interp`, and immediately give a name to the store and value we get as a result
- Updating all the other cases is similar
 - See in-class Racket if time permits

- We chose to evaluate the left side before the right

- We chose to evaluate the left side before the right
 - This is arbitrary

- We chose to evaluate the left side before the right
 - This is arbitrary
 - Could have instead

- We chose to evaluate the left side before the right
 - This is arbitrary
 - Could have instead
- The stores we pass reflects the order

- We chose to evaluate the left side before the right
 - This is arbitrary
 - Could have instead
- The stores we pass reflects the order
 - `interp` of the left argument produces a value

- We chose to evaluate the left side before the right
 - This is arbitrary
 - Could have instead
- The stores we pass reflects the order
 - `interp` of the left argument produces a value
 - *and* a new store

- We chose to evaluate the left side before the right
 - This is arbitrary
 - Could have instead
- The stores we pass reflects the order
 - `interp` of the left argument produces a value
 - *and* a new store
 - `interp` of the right happens *after* the left

- We chose to evaluate the left side before the right
 - This is arbitrary
 - Could have instead
- The stores we pass reflects the order
 - `interp` of the left argument produces a value
 - *and* a new store
 - `interp` of the right happens *after* the left
 - So it should see the effects of interpreting the left

- We chose to evaluate the left side before the right
 - This is arbitrary
 - Could have instead
- The stores we pass reflects the order
 - `interp` of the left argument produces a value
 - *and* a new store
 - `interp` of the right happens *after* the left
 - So it should see the effects of interpreting the left
 - So it *starts* with the store that left *returned*

Order Example

```
{let1 {b {box 3}}  
      {+ {seq {set-box! b {* 3 {unbox b}}}  
          5}  
        {unbox b}}}
```

- The first + argument is a seq expression

Order Example

```
{let1 {b {box 3}}  
  {+ {seq {set-box! b {* 3 {unbox b}}}  
      5}  
    {unbox b}}}
```

- The first + argument is a seq expression
 - Sets the value of b

Order Example

```
{let1 {b {box 3}}  
  {+ {seq {set-box! b {* 3 {unbox b}}}  
      5}  
    {unbox b}}}
```

- The first + argument is a seq expression
 - Sets the value of b
 - Side-effect!

Order Example

```
{let1 {b {box 3}}  
  {+ {seq {set-box! b {* 3 {unbox b}}}  
      5}  
    {unbox b}}}
```

- The first + argument is a seq expression
 - Sets the value of b
 - Side-effect!
- Start with empty store

Order Example

```
{let1 {b {box 3}}  
  {+ {seq {set-box! b {* 3 {unbox b}}}  
      5}  
    {unbox b}}}
```

- The first + argument is a seq expression
 - Sets the value of b
 - Side-effect!
- Start with empty store
- Then store 3 at b's location

Order Example

```
{let1 {b {box 3}}  
      {+ {seq {set-box! b {* 3 {unbox b}}}  
          5}  
      {unbox b}}}
```

- The first + argument is a seq expression
 - Sets the value of b
 - Side-effect!
- Start with empty store
- Then store 3 at b's location
- Then interp {seq {set-box! b {* 3 {unbox b}}}
5}

Order Example

```
{let1 {b {box 3}}  
      {+ {seq {set-box! b {* 3 {unbox b}}}  
          5}  
      {unbox b}}}
```

- The first + argument is a seq expression
 - Sets the value of b
 - Side-effect!
- Start with empty store
- Then store 3 at b's location
- Then interp {seq {set-box! b {* 3 {unbox b}}}
5}
 - Store starts with b := 3

Order Example

```
{let1 {b {box 3}}  
      {+ {seq {set-box! b {* 3 {unbox b}}}  
          5}  
      {unbox b}}}
```

- The first + argument is a seq expression
 - Sets the value of b
 - Side-effect!
- Start with empty store
- Then store 3 at b's location
- Then interp {seq {set-box! b {* 3 {unbox b}}}
5}
 - Store starts with b := 3
 - Unboxing b gives 3, so 9 written as new value for b

Order Example

```
{let1 {b {box 3}}  
      {+ {seq {set-box! b {* 3 {unbox b}}}  
          5}  
      {unbox b}}}
```

- The first + argument is a seq expression
 - Sets the value of b
 - Side-effect!
- Start with empty store
- Then store 3 at b's location
- Then interp {seq {set-box! b {* 3 {unbox b}}}
5}
 - Store starts with b := 3
 - Unboxing b gives 3, so 9 written as new value for b
 - seq throws away the *value* from set-box! and produces 5

Order Example

```
{let1 {b {box 3}}  
      {+ {seq {set-box! b {* 3 {unbox b}}}  
          5}  
      {unbox b}}}
```

- The first + argument is a seq expression
 - Sets the value of b
 - Side-effect!
- Start with empty store
- Then store 3 at b's location
- Then interp {seq {set-box! b {* 3 {unbox b}}}
5}
 - Store starts with b := 3
 - Unboxing b gives 3, so 9 written as new value for b
 - seq throws away the *value* from set-box! and produces 5
- Then interp {unbox b}

Order Example

```
{let1 {b {box 3}}  
      {+ {seq {set-box! b {* 3 {unbox b}}}  
          5}  
      {unbox b}}}
```

- The first + argument is a seq expression
 - Sets the value of b
 - Side-effect!
- Start with empty store
- Then store 3 at b's location
- Then interp {seq {set-box! b {* 3 {unbox b}}} 5}
 - Store starts with b := 3
 - Unboxing b gives 3, so 9 written as new value for b
 - seq throws away the *value* from set-box! and produces 5
- Then interp {unbox b}
 - *starts with new store* which has b := 9

Order Example

```
{let1 {b {box 3}}  
      {+ {seq {set-box! b {* 3 {unbox b}}}  
          5}  
      {unbox b}}}
```

- The first + argument is a seq expression
 - Sets the value of b
 - Side-effect!
- Start with empty store
- Then store 3 at b's location
- Then interp {seq {set-box! b {* 3 {unbox b}}} 5}
 - Store starts with b := 3
 - Unboxing b gives 3, so 9 written as new value for b
 - seq throws away the *value* from set-box! and produces 5
- Then interp {unbox b}
 - *starts with new store* which has b := 9
 - Unboxing b gives 9

Order Example

```
{let1 {b {box 3}}  
      {+ {seq {set-box! b {* 3 {unbox b}}}  
          5}  
      {unbox b}}}
```

- The first + argument is a seq expression
 - Sets the value of b
 - Side-effect!
- Start with empty store
- Then store 3 at b's location
- Then interp {seq {set-box! b {* 3 {unbox b}}} 5}
 - Store starts with b := 3
 - Unboxing b gives 3, so 9 written as new value for b
 - seq throws away the *value* from set-box! and produces 5
- Then interp {unbox b}
 - *starts with new store* which has b := 9
 - Unboxing b gives 9
- Final result 14

Boxes as Values

- We represent boxes using locations in the store

Boxes as Values

- We represent boxes using locations in the store
 - What value is in the box depends on the current store

Boxes as Values

- We represent boxes using locations in the store
 - What value is in the box depends on the current store

Boxes as Values

- We represent boxes using locations in the store
 - What value is in the box depends on the current store

```
(define-type Value
  (numV [num : Number])
  (boolV [bool : Boolean])
  (closureV [arg : Symbol]
            [body : Exp]
            [env : Env])
  (boxV [loc : Location]))
```

Interpreting Box

Interpreting Box

```
(define (interp [env : Env]
                [e : Exp]
                [sto : Store]) : Result
  (type-case Exp e
    ...
    [(boxE a)
     (with [(v sto-v) (interp env a sto)]
       (let ([l (new-loc sto-v)])
         (v*s (boxV loc)
              (override-store loc v
                              sto-v))))))]
```

- Interpret the value we're putting in the box

Interpreting Box

```
(define (interp [env : Env]
                [e : Exp]
                [sto : Store]) : Result
  (type-case Exp e
    ...
    [(boxE a)
     (with [(v sto-v) (interp env a sto)]
       (let ([l (new-loc sto-v)])
         (v*s (boxV loc)
              (override-store loc v
                              sto-v))))))]
```

- Interpret the value we're putting in the box
 - And get the store from this interpretation

Interpreting Box

```
(define (interp [env : Env]
                [e : Exp]
                [sto : Store]) : Result
  (type-case Exp e
    ...
    [(boxE a)
     (with [(v sto-v) (interp env a sto)]
       (let ([l (new-loc sto-v)])
         (v*s (boxV loc)
              (override-store loc v
                              sto-v))))))]
```

- Interpret the value we're putting in the box
 - And get the store from this interpretation
- Create a new store that has the value at a new address

Interpreting Box

```
(define (interp [env : Env]
                [e : Exp]
                [sto : Store]) : Result
  (type-case Exp e
    ...
    [(boxE a)
     (with [(v sto-v) (interp env a sto)]
       (let ([l (new-loc sto-v)])
         (v*s (boxV loc)
              (override-store loc v
                              sto-v))))))]
```

- Interpret the value we're putting in the box
 - And get the store from this interpretation
- Create a new store that has the value at a new address
- Return the box with the new location

Interpreting Box

```
(define (interp [env : Env]
                [e : Exp]
                [sto : Store]) : Result
  (type-case Exp e
    ...
    [(boxE a)
     (with [(v sto-v) (interp env a sto)]
       (let ([l (new-loc sto-v)])
         (v*s (boxV loc)
              (override-store loc v
                              sto-v))))))]
```

- Interpret the value we're putting in the box
 - And get the store from this interpretation
- Create a new store that has the value at a new address
- Return the box with the new location
 - Return store is the store with the new location and value

Interpreting Unbox

```
[(unboxE a)
 (with [(v sto-v) (interp env a sto)]
  (type-case Value v
    [(boxV l) (v*s (lookup l sto-v)
                    sto-v)]
    [else (error 'interp "not a box")])))]
```

- Interpret the expression to a value

Interpreting Unbox

```
[(unboxE a)
 (with [(v sto-v) (interp env a sto)]
  (type-case Value v
    [(boxV l) (v*s (lookup l sto-v)
                    sto-v)]
    [else (error 'interp "not a box")])))]
```

- Interpret the expression to a value
 - Get the value and the store from this, since it might have had side effects

Interpreting Unbox

```
[(unboxE a)
 (with [(v sto-v) (interp env a sto)]
  (type-case Value v
    [(boxV l) (v*s (lookup l sto-v)
                    sto-v)]
    [else (error 'interp "not a box")])))]
```

- Interpret the expression to a value
 - Get the value and the store from this, since it might have had side effects
- Check if the value is a Box

Interpreting Unbox

```
[(unboxE a)
 (with [(v sto-v) (interp env a sto)]
  (type-case Value v
    [(boxV l) (v*s (lookup l sto-v)
                    sto-v)]
    [else (error 'interp "not a box")])))]
```

- Interpret the expression to a value
 - Get the value and the store from this, since it might have had side effects
- Check if the value is a Box
 - Type error otherwise

Interpreting Unbox

```
[(unboxE a)
 (with [(v sto-v) (interp env a sto)]
  (type-case Value v
    [(boxV l) (v*s (lookup l sto-v)
                    sto-v)]
    [else (error 'interp "not a box")])))]
```

- Interpret the expression to a value
 - Get the value and the store from this, since it might have had side effects
- Check if the value is a Box
 - Type error otherwise
- Get its location, and produce whatever value was at that location

Interpreting Unbox

```
[(unboxE a)
 (with [(v sto-v) (interp env a sto)]
  (type-case Value v
    [(boxV l) (v*s (lookup l sto-v)
                    sto-v)]
    [else (error 'interp "not a box")])))]
```

- Interpret the expression to a value
 - Get the value and the store from this, since it might have had side effects
- Check if the value is a Box
 - Type error otherwise
- Get its location, and produce whatever value was at that location
 - With the store from evaluating the box

Interpreting Set-box!

Interpreting Set-box!

```
[(set-box!E bx val)
 (with [(v-b sto-b) (interp bx env sto)]
  (with [(v-v sto-v) (interp val env sto-b)]
   (type-case Value v-b
    [(boxV loc)
     (v*s v-v
      (override-store loc v-v
                       sto-v))])
    [else (error 'interp "not a box")]))))]
```

- Interpret the box to a value

Interpreting Set-box!

```
[(set-box!E bx val)
 (with [(v-b sto-b) (interp bx env sto)]
  (with [(v-v sto-v) (interp val env sto-b)]
   (type-case Value v-b
    [(boxV loc)
     (v*s v-v
      (override-store loc v-v
                       sto-v))])
    [else (error 'interp "not a box")]))))]
```

- Interpret the box to a value
 - Get the store from this execution

Interpreting Set-box!

```
[(set-box!E bx val)
 (with [(v-b sto-b) (interp bx env sto)]
  (with [(v-v sto-v) (interp val env sto-b)]
   (type-case Value v-b
    [(boxV loc)
     (v*s v-v
      (override-store loc v-v
                       sto-v))])
    [else (error 'interp "not a box")]))))]
```

- Interpret the box to a value
 - Get the store from this execution
- Interpret the expression to be stored to a value

Interpreting Set-box!

```
[(set-box!E bx val)
 (with [(v-b sto-b) (interp bx env sto)]
  (with [(v-v sto-v) (interp val env sto-b)]
   (type-case Value v-b
    [(boxV loc)
     (v*s v-v
      (override-store loc v-v
                       sto-v))])
    [else (error 'interp "not a box")]))))]
```

- Interpret the box to a value
 - Get the store from this execution
- Interpret the expression to be stored to a value
 - In the store we previously computed, getting a new store

Interpreting Set-box!

```
[(set-box!E bx val)
  (with [(v-b sto-b) (interp bx env sto)]
    (with [(v-v sto-v) (interp val env sto-b)]
      (type-case Value v-b
        [(boxV loc)
         (v*s v-v
              (override-store loc v-v
                              sto-v)))]
        [else (error 'interp "not a box")]))))]
```

- Interpret the box to a value
 - Get the store from this execution
- Interpret the expression to be stored to a value
 - In the store we previously computed, getting a new store
- Check that the box value is actually a box

Interpreting Set-box!

```
[(set-box!E bx val)
  (with [(v-b sto-b) (interp bx env sto)]
    (with [(v-v sto-v) (interp val env sto-b)]
      (type-case Value v-b
        [(boxV loc)
         (v*s v-v
              (override-store loc v-v
                              sto-v))])
        [else (error 'interp "not a box")]))))]
```

- Interpret the box to a value
 - Get the store from this execution
- Interpret the expression to be stored to a value
 - In the store we previously computed, getting a new store
- Check that the box value is actually a box
- Return the other value, *in the new store with the box location overwritten*

Interpreting Set-box!

```
[(set-box!E bx val)
  (with [(v-b sto-b) (interp bx env sto)]
    (with [(v-v sto-v) (interp val env sto-b)]
      (type-case Value v-b
        [(boxV loc)
          [(boxV loc)
            (v*s v-v
              (override-store loc v-v
                            sto-v))]]
        [else (error 'interp "not a box")]])))]
```

- Interpret the box to a value
 - Get the store from this execution
- Interpret the expression to be stored to a value
 - In the store we previously computed, getting a new store
- Check that the box value is actually a box
- Return the other value, *in the new store with the box location overwritten*
 - Could have also returned dummy value 0, #t, etc.

Interpreting Set-box!

```
[(set-box!E bx val)
  (with [(v-b sto-b) (interp bx env sto)]
    (with [(v-v sto-v) (interp val env sto-b)]
      (type-case Value v-b
        [(boxV loc)
          [(boxV loc)
            (v*s v-v
              (override-store loc v-v
                            sto-v))]]
        [else (error 'interp "not a box")]])))]
```

- Interpret the box to a value
 - Get the store from this execution
- Interpret the expression to be stored to a value
 - In the store we previously computed, getting a new store
- Check that the box value is actually a box
- Return the other value, *in the new store with the box location overwritten*
 - Could have also returned dummy value $\emptyset$, $\#t$, etc.
 - Purpose is the side effect

Interpreting Seq

```
[(seqE l r)
 (with [(v-l sto-l) (interp env l sto)]
  (interp env r sto-l))]
```

- Evaluate the first expression *but do nothing with the value*

Interpreting Seq

```
[(seqE l r)
 (with [(v-l sto-l) (interp env l sto)]
  (interp env r sto-l))]
```

- Evaluate the first expression *but do nothing with the value*
 - Just get the store that's the result of evaluating it

Interpreting Seq

```
[(seqE l r)
 (with [(v-l sto-l) (interp env l sto)]
  (interp env r sto-l))]
```

- Evaluate the first expression *but do nothing with the value*
 - Just get the store that's the result of evaluating it
- Evaluate the second expression *in the store the first returned*

Stores vs Environment

- Environments enable *static scope*

Stores vs Environment

- Environments enable *static scope*
 - Follow the stack like structure of function calls

Stores vs Environment

- Environments enable *static scope*
 - Follow the stack like structure of function calls
 - When we return from a function call, we keep evaluating in the old environment

Stores vs Environment

- Environments enable *static scope*
 - Follow the stack like structure of function calls
 - When we return from a function call, we keep evaluating in the old environment
- Stores are inherently dynamic

Stores vs Environment

- Environments enable *static scope*
 - Follow the stack like structure of function calls
 - When we return from a function call, we keep evaluating in the old environment
- Stores are inherently dynamic
 - We *want* them to always take the value from the point unbox is executed

Stores vs Environment

- Environments enable *static scope*
 - Follow the stack like structure of function calls
 - When we return from a function call, we keep evaluating in the old environment
- Stores are inherently dynamic
 - We *want* them to always take the value from the point unbox is executed
 - Linearly threaded through the program

Stores vs Environment

- Environments enable *static scope*
 - Follow the stack like structure of function calls
 - When we return from a function call, we keep evaluating in the old environment
- Stores are inherently dynamic
 - We *want* them to always take the value from the point unbox is executed
 - Linearly threaded through the program
 - Once we've updated a store (by generating a new one), we *never use the old store again*

Stores vs Environment

- Environments enable *static scope*
 - Follow the stack like structure of function calls
 - When we return from a function call, we keep evaluating in the old environment
- Stores are inherently dynamic
 - We *want* them to always take the value from the point unbox is executed
 - Linearly threaded through the program
 - Once we've updated a store (by generating a new one), we *never use the old store again*
 - ... unless we implement more advanced features e.g. undo, backtracking